



OPTIMIZATION TECHNIQUES

Shivam Gaind

What is Optimization?

- Start with an initial design
- Define a goal (ex. maximize gain, minimize weight)
- Add constraints (ex. size limits, material limits)
- Run loop (design is continuously improved)
- Stops once convergence occurs (ex. gain is no longer improved, weight is no longer reduced)

What is Parameter Sweep?

- Often used before Optimization
- Choose a parameter (permittivity)
- Test different values
- Goal is to identify trends within the material (plot gain vs. permittivity)

Local Optimization

- Start with an initial design
- Calculates value for goal (ex. gain, weight)
- Changes parameter that improves it
- Repeats until local optimum has been reached (value is no longer improved)

Global Optimization (slow)

- Uses multiple designs
- Best designs are kept
- New designs are created by crossing over best designs
- Loop runs until improvement becomes marginal

Topology Optimization

- Starts with a single design
- Design is continuously improved
- Produce a single final optimized design



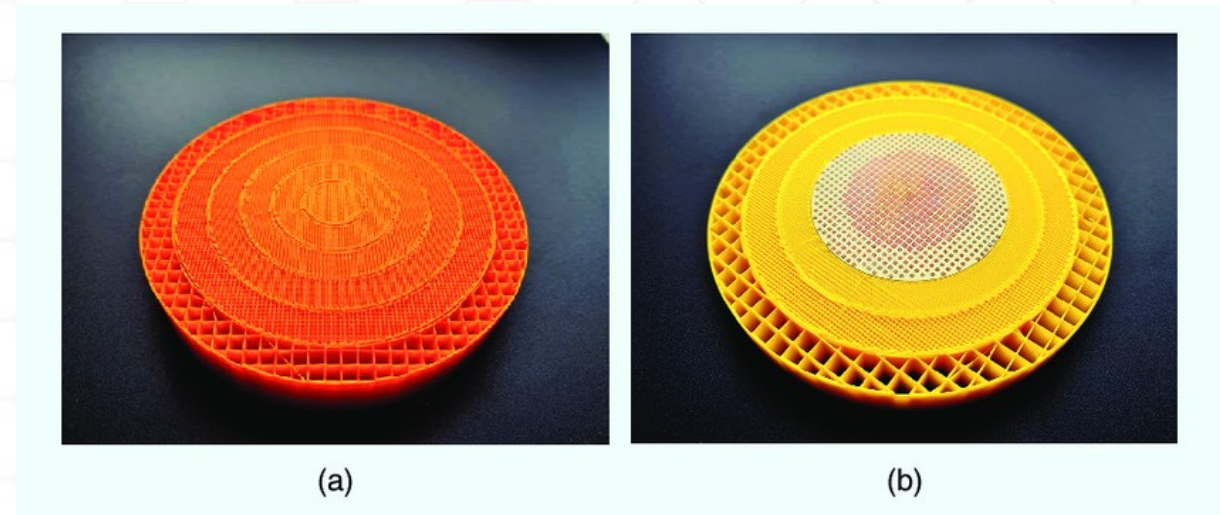
Articles

General Notes

- Introduces a new way to design lenses for antennas that improves gain without making the antennas larger
- GRIN Lenses are made from a single material but the internal structure changes.
- Variations of structure within the lens are used to

Key Findings

- Result: The GRIN lens antenna system performed similarly to a 20 dB horn antenna (much larger in size)

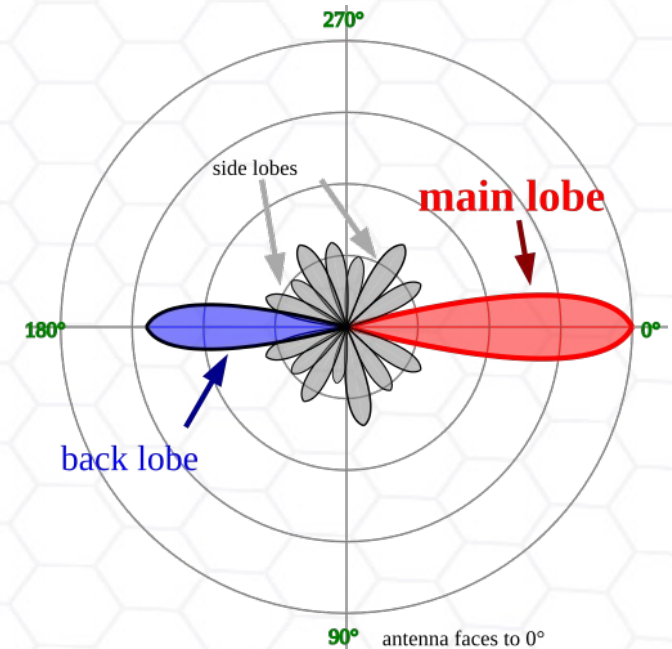


What is Gain?

- Gain (dB) is a measure of how strong the antennas signal is
 - 0 dB: spreads power equally in all directions (theoretical)
 - 10 dB: 10× stronger in one direction
 - 20 dB: 100× stronger in one direction

Benefits of GRIN Lenses

- Increases gain (creates a stronger and more focused signal)
- Reduces sidelobes
- Makes the antenna smaller

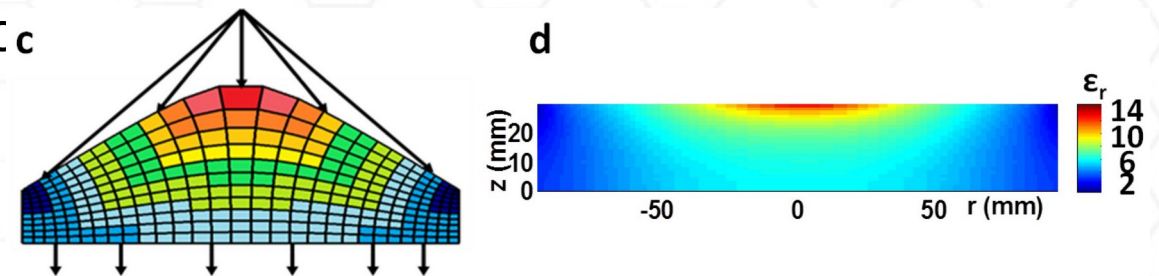


How are GRIN lenses designed?

- Transformation Optics
(used for designing 2D GRIN Lenses)
 - Start with an existing lens
 - Apply a coordinate transformation so that the electromagnetic waves behave as if space is bent
 - Derive the material properties that you need from that equation
 - Build the lens using those material properties

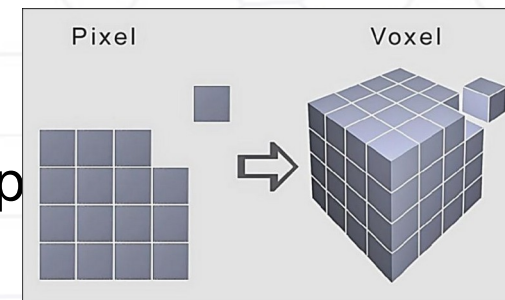
Limitations

- Often produces anisotropic materials, or materials with extremely high permittivity values
- We want GRIN lenses to be easily fabricate



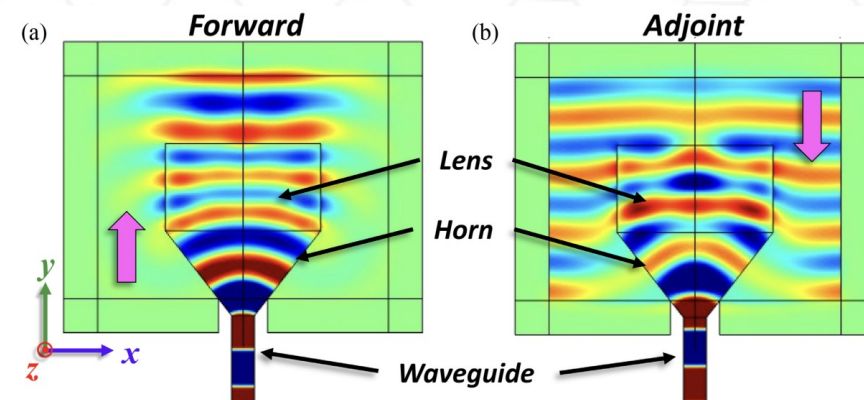
How do voxels change an object?

- Tweak voxel
- Re-run simulation
- Repeat for each voxel (thousands of simulations end up run)



How are GRIN lens designed?

- Instead of TO GRIN lenses can be designed using **Adjoint Optimization**
- Adjoint Optimization uses only two simulations to see how each voxel changes
- **Forward Simulation**
 - Antenna is turned on
 - Waves pass through the lens
 - Measure the gain of the beam
- **Adjoint Simulation**
 - Its flipped



• A virtual electromagnetic wave toward the antenna

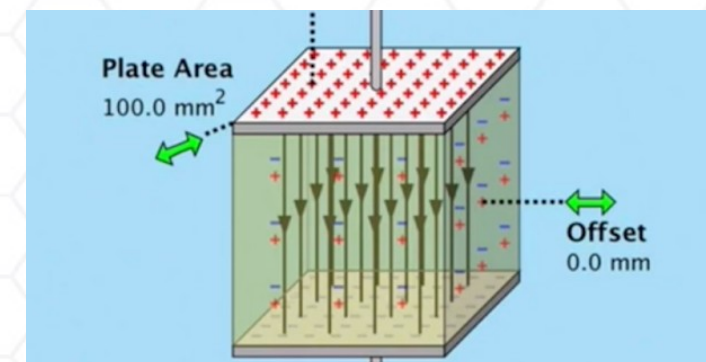
Adjoint Sensitivity Optimization of Three-Dimensional Directivity-Enhancing, Size-Reducing GRIN Lenses

Gradient

- By multiplying the **forward field x adjoint field** you get the **gradient**
 - Forward field is the actual radiated field (Volts/Meter)
 - Adjoint field is the backward propagating field (Volts/Meter)

Permittivity

- Positive gradient -> Increase permittivity here**
- Negative gradient -> Decrease permittivity here**
- Near Zero gradient -> No change needed here**
- More dielectric material = higher permittivity
- You decrease the permittivity by adding more air (voids)

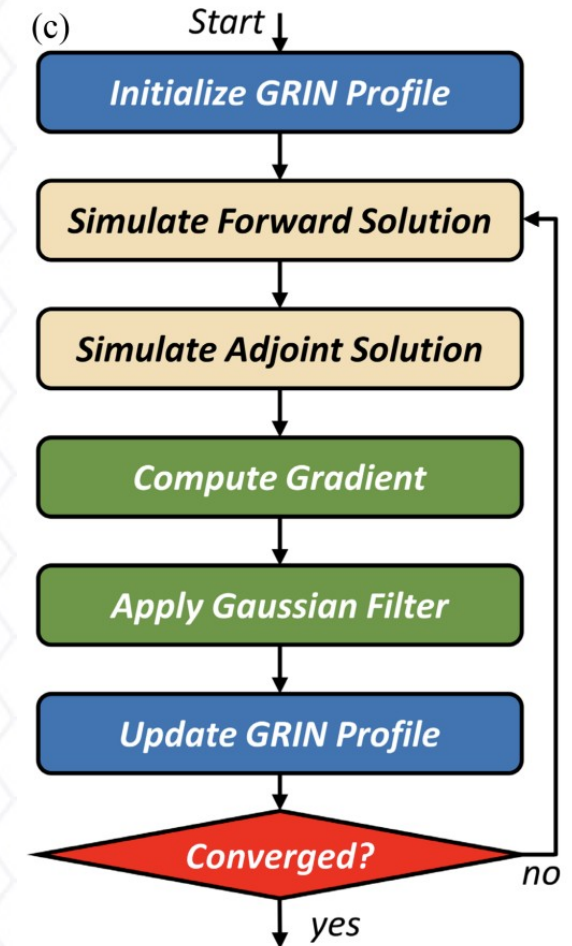


Permittivity describes how a material responds to a electric field

High permittivity -> electromagnetic waves travel

Optimization Loop (the algorithm)

1. Start with initial GRIN lens profile (starting permittivity values for each voxel)
2. Run Forward Simulation
3. Run Adjoint Simulation
4. *(Both simulations are run on COMSOL Multiphysics)
5. Solve for gradient of each voxel
6. Apply a gaussian filter (replaces each voxel value with weighted average of its neighbors)
 - Closer voxels influence it more
 - Further out voxels influence it less
7. Apply permittivity bounds
 - if any values are outside the allowable permittivity range, set them to the maximum or minimum allowable value
8. Repeat the optimization loop until the gain stops improving

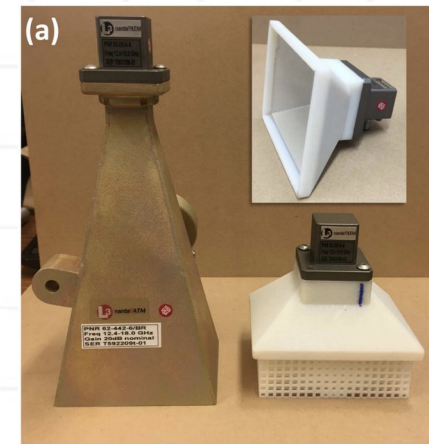
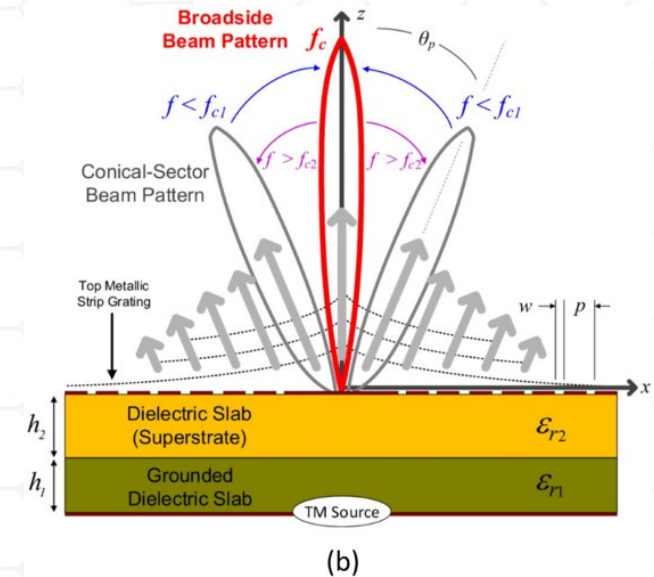


What is being Optimized?

- The gain (strength) of the broadside beam is being maximized
- The antenna should be able to send as much power possible straight forward at the 0 degree angle

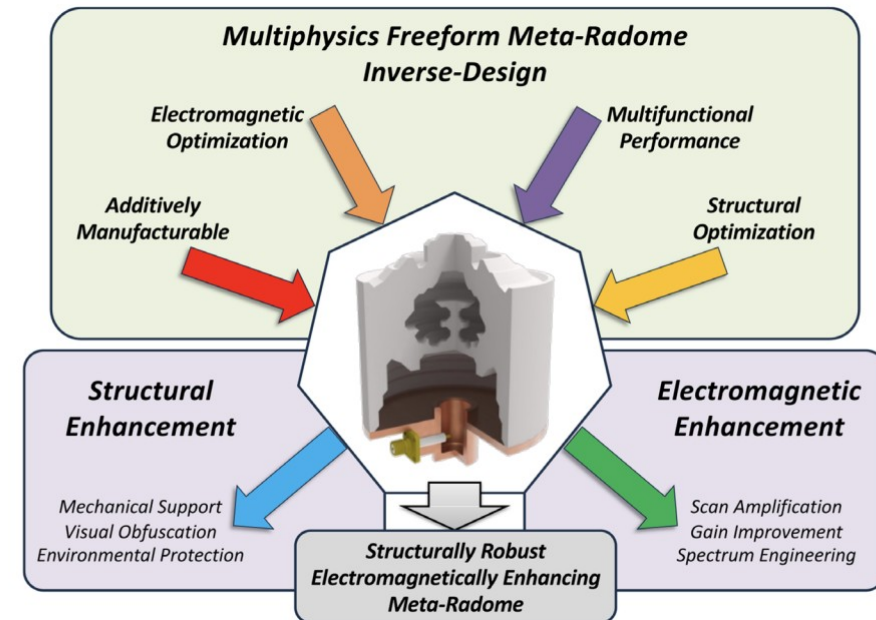
Design & Result

- GRIN Lens was printed using Stratasys 3D printer in plastic material
- Printed different size holes within each voxel
(allows control over how much air enters each voxel)
- Result was a horn that was 60% smaller and 40% lighter



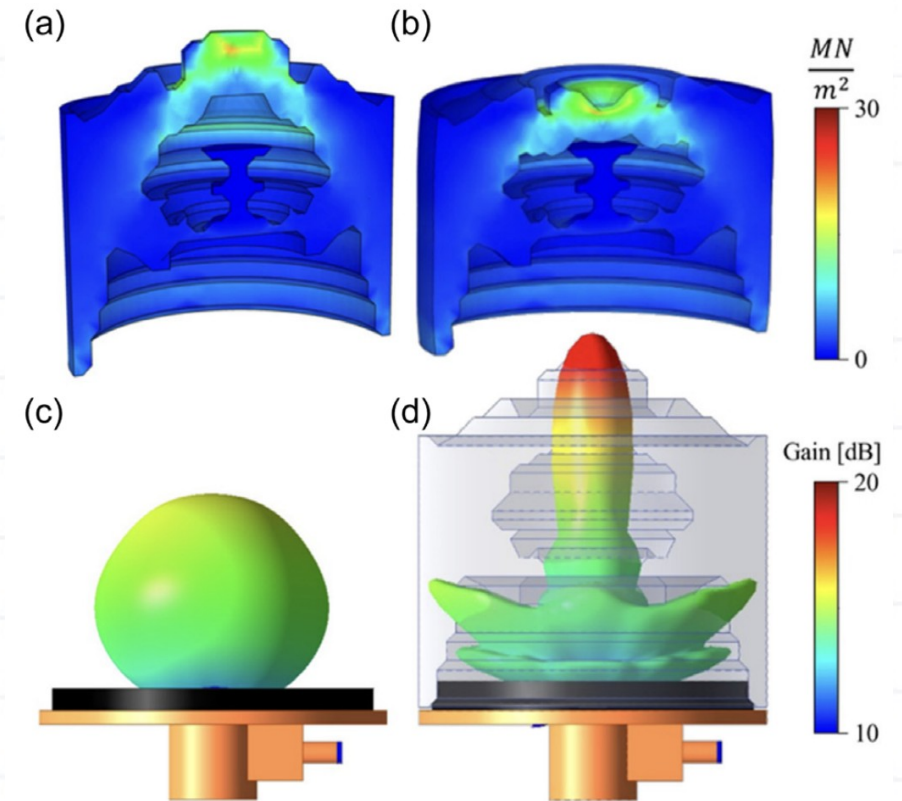
General Notes

- A radome is a cover that protects an antenna against environmental conditions
- Goal is to design a meta-radome that:
 - Improves antenna gain
 - Is easily manufacturable
 - Can survive 4400N mechanical load
- Multiphysics Optimization is used (involves concepts from EM & mechanical design)



Electromagnetic Optimization

- Goal is to maximize gain coming from the broadside beam (image d)
- Adjoint optimization (same technique used for Grin Lenses)
 1. Convert design into voxels
 - 2. Run Forward Simulation** (provides gain, this shows how antenna is currently performing)
 - 3. Run Adjoint Simulation** (inject wave at 0 degrees since broadside beam is where we want to maximize gain)
 4. Solve gradient (**forward field x adjoint field**)
 - Positive gradient -> add more material (increase permittivity)
 - Negative gradient -> Decrease





Mechanical Optimization (topology optimization)

- Goal is to solve for Compliance
- Compliance measures how much the material bends
- Compliance = Force / Displacement
- Lower compliance = stiffer material
- Displacement is solved using the stiffness matrix

**U = Displacement
Vectors**

(it tells us how much each
voxel moved in the x, y,
and z directions)

$U = [x, y, z]$

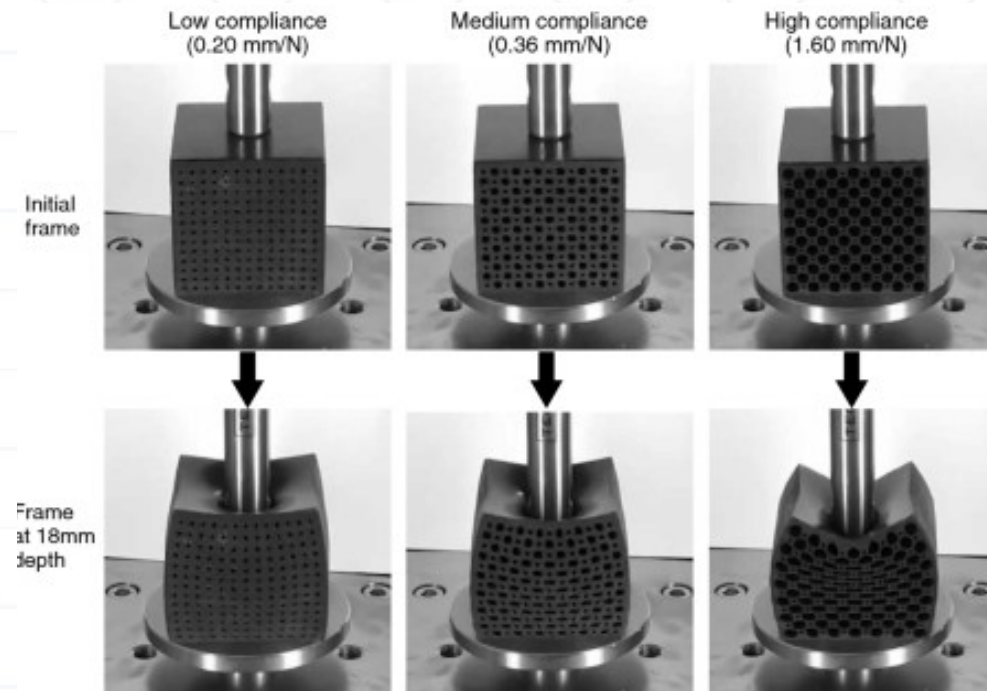
K = global stiffness matrix
of the entire structure

Each row corresponds to
one DOF force equation

$$c = \frac{1}{2} U^T K U$$

$$K U = F$$

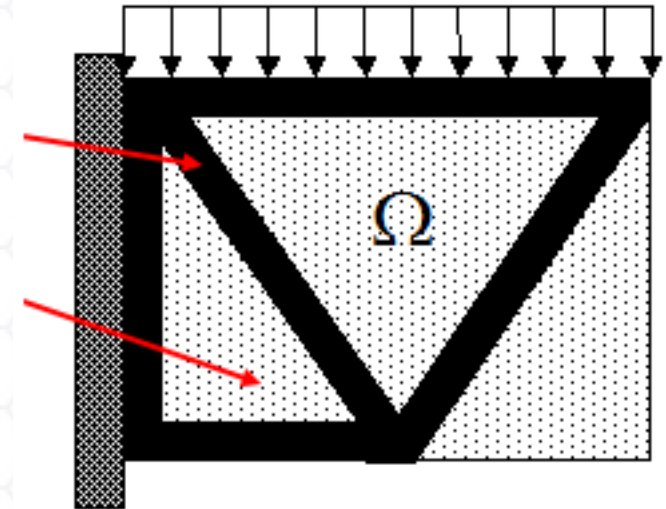
$$\rho_i \in [0,1], \forall i$$



SIMP model

- The stiffness of each voxel is expressed using:
- $K_E = \rho^p KE$
- K_E = Stiffness of each voxel
- KE = Stiffness of the solid material
- ρ = density (0 to 1)
- p = penalization factor

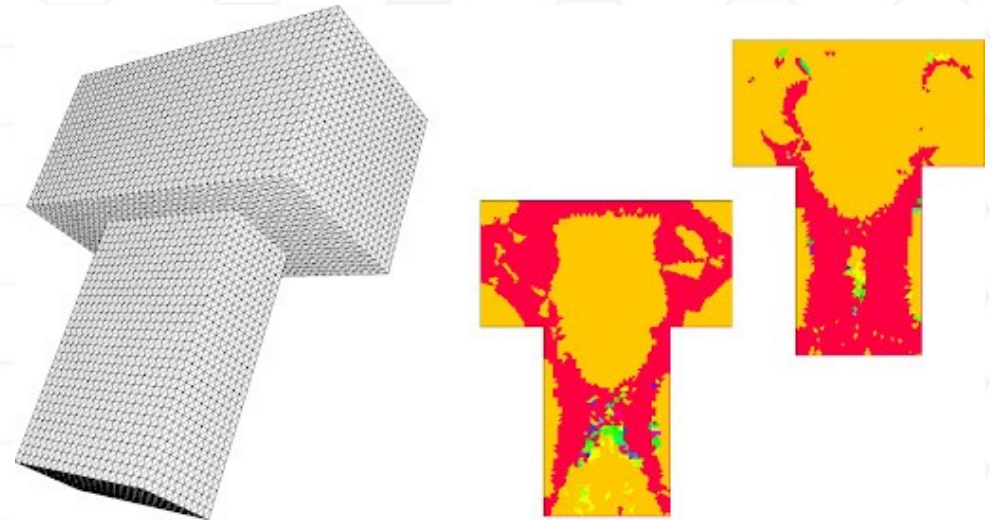
Penalization factor (p) : it makes intermediate (half) density voxels perform poorly so that during optimization they are pushed to either be fully solid or empty.



Density (ρ) $\uparrow \Rightarrow$ Elasticity of each voxel (E) $\uparrow \Rightarrow$ Stiffness (k) $\uparrow \Rightarrow$ Displacement (x) $\downarrow \Rightarrow$ Compliance(C) $\downarrow$

SIMP model

- The **SIMP** model (Solid Isotropic Material with Penalization)
 - The material is divided into many voxels
 - Each voxel has density value (ρ) between 0 and 1
 - $\rho = 0$ void (no material)
 - $\rho = 1$ (solid)
- **The voxels in red are the “removed” material; their density was driven to zero during the SIMP optimization process.**



Combining Electromagnetic & Mechanical Optimization

- For each voxel we look at
 - Does adding material improve gain? ($G_M = \Delta \text{Gain} / \Delta \text{Density}$)
 - Does adding material improve stiffness? ($G_{ME} = \Delta \text{Stiffness} / \Delta \text{Density}$)
- **We combine the gradients using a weighted sum**
$$G = (1 - \alpha_r) G_{ME} + \alpha_r G_{EM}$$
- G_M = gradient from gain
- G_{ME} = gradient from compliance
- $\alpha = .7$ (weighting factor)
- **We get one final gradient for each voxel (G)**
 - Positive gradient -> add more material
 - Negative gradient -> reduce material

Optimization Loop (the algorithm)

1. Run forward EM simulation
2. Run adjoint EM simulation
3. Run mechanical simulation
4. Compute gradients
5. Combine gradients
6. Update voxel densities

*This whole process took about 7 minutes to generate a design

We keep running this optimization loop until the gain no longer increases and the compliance no longer significantly decreases. This tells us that we have converged to a local optimum.

99 - Line Topology Optimization Code

- This is the original minimal MATLAB code written by Ole Sigmund (2001)
- Solves the problem “Where should you place material to make a structure as stiff as possible (given a fixed amount of material)?”
- Used for 2-D Optimization

88-Line Version (Vectorized, Faster)

Provides the same result as the 99 line version

```

1 %%% A 99 LINE TOPOLOGY OPTIMIZATION CODE BY OLE
  SIGMUND, OCTOBER 1999 %%%
2 function top(nelx,nely,volfrac,penal,rmin);
3 % INITIALIZE
4 x(1:nely,1:nelx) = volfrac;
5 loop = 0;
6 change = 1.;
7 % START ITERATION
8 while change > 0.01
9     loop = loop + 1;
10    xold = x;
11    % FE- ANALYSIS
12    [U]=FE(nelx,nely,x,penal);
13    % OBJECTIVE FUNCTION AND SENSITIVITY ANALYSIS
14    [KE] = 1k;
15    c = 0.;
16    for ely = 1:nely
17        for elx = 1:nelx
18            n1 = (nely+1)*(elx-1)+ely;
19            n2 = (nely+1)*elx+ely;
20            Ue = U([2*n1-1;2*n1; 2*n2-1;2*n2; 2*n2+1;
21                    2*n2+2; 2*n1+1;2*n1+2],1);
21            c = c + x(ely,elx)^penal*Ue'*KE*Ue;
22            dc(ely,elx) = -penal*x(ely,elx)^(penal-1)*
23                           Ue'*KE*Ue;

```

The optimization process

- 1) Starts with a block of material
- 2) Applies loads and supports
- 3) Uses Finite Element Analysis to solve for deformation
- 4) Solves for which areas carry stress
- 5) Gets rid of inefficient material
- 6) This loop runs until you get an optimized shape

99 - Line Topology Optimization Code **Main Sections**

- 7) Input parameters
- 8) Initialize design variables
- 9) Optimization loop
- 10) Finite Element Analysis
- 11) Sensitivity Analysis
- 12) Filtering

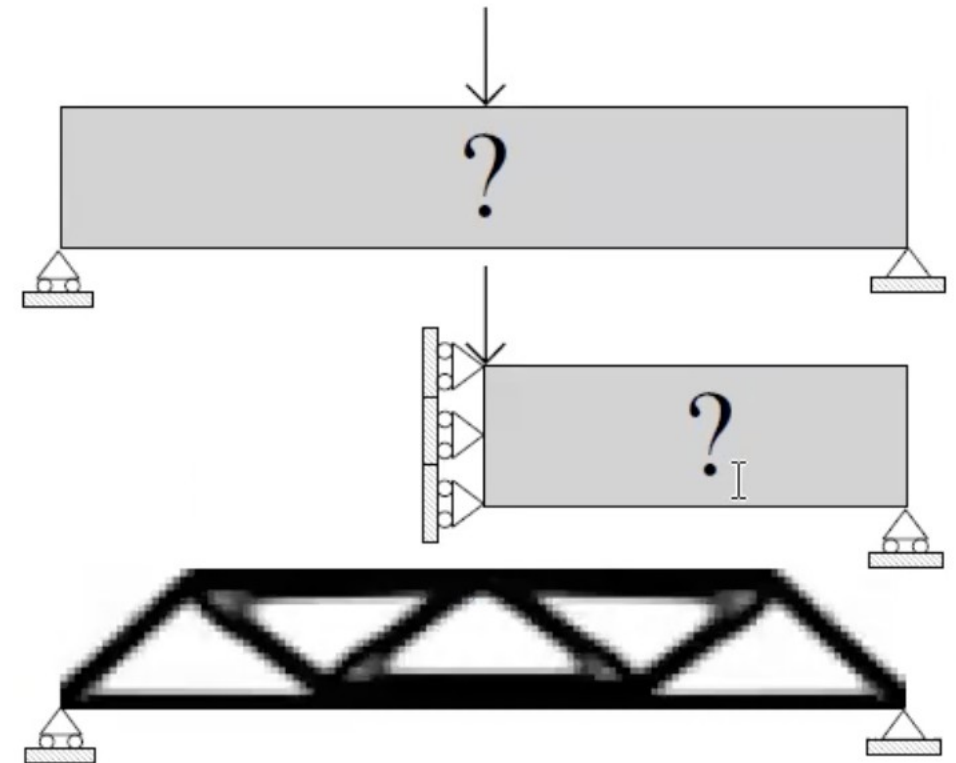


Fig. 1 Topology optimization of the MBB-beam. Top: full design domain, middle: half design domain with symmetry boundary conditions and bottom: resulting topology optimized beam (both halves)

function top(nelx,nely,volfrac,penal,rmin)

- 1) First step is to create a function that takes 5 input parameters
- 2) Design grid size = $nelx * nely$
 - nelx: number of elements in the x direction
 - nely: number of elements in the y direction
- 3) Volume Fraction Constant (volfrac)
 - Tells you how much of the design grid can have material
 - Ex. volfrac = .5, if we have 1000 elements only 500 elements worth of material is allowed
- 4) Penalization factor (usually 3): forces design to become solid (1) or void (0)
- 5) Rmin: Radius used for the filter (replaces each element's value with the weighted average of its neighbors)
 - Ex. $Rmin = 1.5$ each element is influenced by others within radius of 1.5

Initialize design variables

- 1) Tell the optimizer that all elements start with the same density
 - `x = volfrac * ones(nely,nelx);`
- 2) Set the optimization loop to run until the design stops changing significantly
 - `while change > 0.01`
`loop = loop + 1;`
`xold = x;`

```
function top(nelx,nely,volfrac,penal,rmin)

% INITIALIZE
x = volfrac*ones(nely,nelx);
loop = 0;
change = 1;

while change > 0.01
    loop = loop + 1;
    xold = x;
```

Finite Element Analysis (FEA)

A.) Build the element stiffness matrix

- $KE = lk;$
- Calls the function lk , which builds an 8x8 base element stiffness matrix
- This is the base element stiffness matrix, assuming it is fully solid

B.) Each element has a stiffness matrix

- $K_E = \rho^p KE$
- KE is the 8x8 base element stiffness matrix

C.) Build the global stiffness matrix (88 line version)

- $K = \text{sparse}(ik, jk, sk)$
- Built-in function that combines all of the element stiffness matrices into one matrix.
 - $iK(n)$: which row
 - $jK(n)$: which column
 - $sK(n)$: (K_E) what stiffness value goes there
- Sparse creates huge matrices and saves memory

Finite Element Analysis (FEA)

D.) Solve for displacement matrix U

(gives you the displacement vector of each node)

`U(freedofs) = K(freedofs,freedofs) \ F(freedofs);`

- Originally, we had the global stiffness matrix for the entire design grid.
- Now, we only keep the matrix corresponding to the degrees of freedom that are free.
- We set its size to: $\text{Size} = (\# \text{ free DOFs}) \times (\# \text{ free DOFs})$, so that we can extract a smaller matrix from the full global stiffness matrix K.
- With F(freedofs), we take our force vector and reduce it down to only the entries corresponding to the free degrees of freedom.

Compliance

A.) Solve for the compliance of each element and use that to find the total compliance of the structure

$$c_e(e) = U_e^T K_0 U_e$$

ce =
sum((U(edofMat)*KE).*U(edofMat),2)

- KE: base element stiffness matrix
- **edofMat** = (number of elements) * 8, each element has 8 DOF corresponding to it
- **U(edofMat)**: Takes all the rows listed in edofMat

B.) Solve for the total compliance of the entire structure

c =
sum(sum((x.^penal).*ce));

c.) Solve for the gradient of compliance with respect to density

dc = -penal * x.^(penal-1) .*
ce;

- Positive gradient -> add more material
- Negative gradient -> reduce